编译器设计文档

彭子峻 23373334

目录

1 参考设计介绍 **3**
1.1 总体结构 3
1.2 接口设计 3
1.3 文件组织 4

2 编译器总体设计 **4**
2.1 总体结构 4
2.2 接口设计 5
2.3 文件组织 6

3 词法分析 **7**
3.1 总体设计 7
3.2 方法实现 8
3.3 编码后的修改 9

4 语法分析 **9**
4.1 总体设计 9
4.2 方法实现 9
4.3 编码后的修改 10

5 语义分析 **10**
5.1 总体设计 11
5.2 编码后的修改 11

6 代码生成 **12**
6.1 LLVM IR的生成 12
6.1.1 编码后的修改 13
6.2 MIPS汇编的生成 13
6.2.1 编码后的修改 14

7 代码优化 **15**
7.1 总体设计 15
7.2 编码后的修改 15

1 参考设计介绍

我选取了参考资料中的pl0-compiler，这是一个使用Pascal语言实现的编译器。

1.1 总体结构

这个编译器遵循了经典的编译流程，并通过生成中间代码并内置解释器的方式，完成最终执行。其主要功能模块和过程如下：

- 词法分析：通过getsym()过程实现。
- 语法分析、语义分析：采用递归下降法完成语法分析，逐层调用block(), statement(), expression()等语法成分的分析process。
  语义分析在语法分析过程中同时完成，存在耦合。
- 代码生成：生成自定义的栈式中间代码。
- 解释执行：通过interpret()过程内置的栈式机，解释执行生成的中间代码。

注意到，在本实现中，词法分析、语法分析、语义分析与代码生成这四个部分，均存在一定程度的耦合。

1.2 接口设计

整个编译器使用面向过程语言编写，不存在接口，大部分情况下使用全局变量传递信息。

比如说，词法分析中，参考实现通过sym，id与num等全局变量，向语法分析过程传递 token 信息。

在语法分析过程中，各语法成分对应的分析过程，其参数列表均有一些相同的参数，某种意义上这也算接口的一种：

Procedure statement(fsys : symset);

Procedure expression(fsys: symset);

Procedure term(fsys : symset);

Procedure factor(fsys : symset);

Procedure expression(fsys: symset);

Procedure condition(fsys : symset);

其中，symset 为该分析过程接受的符号集。

1.3 文件组织

参考编译器是单一文件，不存在文件组织。

2 编译器总体设计

编译器使用Java实现，采用面向对象的程序构造方法进行组织，并利用了Java 17的部分特性。

2.1 总体结构

编译器整体划分为前端、后端两个部分。

前端负责将源程序转换为中间代码，并对中间代码进行变换（优化）。本实现选择LLVM IR作为中间代码。

后端负责将中间代码翻译为MIPS汇编。期间会进行消Phi，物理寄存器分配等过程。

前端部分共执行以下过程：

- 词法分析：产生TokenStream，供语法分析使用；
- 语法分析：产生CompileUnit，即AST的顶层模块；
- 语义分析：将语法树中定义、使用的符号关联起来，并指定符号的数据类型；
- 中间代码生成：通过各语法单元内的build()方法，转换为LLVM IR；
- 中间代码优化：执行各优化有关Pass。

前端中可能出现的错误，通过构建一ErrorEntry类进行管理。前端的词法、语法、语义分析各部分，均有一ErrorEntry容器，收集当前阶段产生的错误。

后端负责以下过程：

- 中间代码翻译：将LLVM IR，翻译为使用虚拟寄存器作为操作数的MIPS汇编；

- 消Phi：正如其名
- 物理寄存器分配：为各虚拟寄存器指派物理寄存器；在此步完成后，整个编译过程可认为结束。
- 后端优化：主要是Peephole。

2.2 接口设计

在设计各个类时，对于属性和方法的可见性，我进行了较为严格的管理。除了跨类交互需要用到的方法设为public外。仅在类内部使用的均设为private。在子类中亦会使用到的，则设置为protected类。

- 词法解析：由frontend/Lexer实现，
  暴露其构造方法，运行分析的analyze()方法，以及
  收集错误，获取/打印TokenStream需要的方法；
- 语法分析：由frontend/Parser开始，
  暴露其构造方法，运行分析的parse()方法，和获取CompileUnit和错误信息的方法；
  各语法成分均创建类，继承ASTNode抽象类，实现静态parse()方法;
  Parser通过自语法成分顶层CompileUnit往下递归语法成分对应的parse()，完成整个语法树的构建。
- 语义分析：由frontend/Tabulator开始，
  暴露构造方法，运行分析的tabulate()方法，以及整个分析过程需要的辅助方法；
  各语法成分类需实现visit()方法，与Tabulator类交互，完成符号关系的构建；
- 中间代码生成：由frontend/IrBuilder开始；
  各语法成分类需实现build()方法，IrBuilder通过从CompileUnit逐层向下递归调用build()方法，完成中间代码的构建；
- 中间代码优化：构建接口frontend/llvm/Pass，要求实现run()方法；主程序通过实例化各Pass进一个有序容器内，随后顺序调用run()方法，完成对中间代码的变形。

- 目标代码生成：由backend/mips/MipsBuilder开始，调用backend/mips/instBuilder中，各种LLVM IR对应的构建方法build()，完成到目标代码的转换。通过backend/mips/process中定义的各个后端Pass，完成消Phi，寄存器分配，后端优化等过程。

2.3 文件组织

- Compiler.java：编译器入口；
- Executer.java：编译器运行主程序，负责按配置控制各编译器过程的运行；
- settings/Settings.java：编译器配置，控制编译器运行哪些过程，文件打印位置等；
- utils/：存放整个编译器中可能需要用到的自定义数据结构；
- frontend/：编译器前端内容
-
  - ./datatype：语义分析、中间代码需用到的类型系统
  - ./error：前端记录错误需用到的数据结构
  - ./symbol：语义分析需用到的符号系统
  - ./syntax：语法分析时的各语法成分
  - ./token：词法分析时，构建的Token有关数据结构
  - ./llvm：LLVM相关逻辑：
  -
    - ./analysis：进行优化前需进行的分析Pass，如控制流分析等
    - ./optimization：各优化Pass
    - ./tools：LLVM IR构建时使用的辅助数据结构
    - ./value：LLVM IR中所有的Value，包括但不限于指令等
    - ./IrModule：LLVM IR构建完成后的顶层模块
  - ./IrBuilder.java：LLVM IR构建过程的控制、辅助类
  - ./Lexer.java：词法分析逻辑
  - ./Parser.java：语法分析的控制、辅助类
  - ./Tabulator.java：语义分析的控制、辅助类

- backend/mips：编译器后端内容
-
  - ./instBuilder：存放将各LLVM IR翻译为MIPS的逻辑
  - ./instruction：MIPS指令定义
  - ./operand：MIPS操作数定义
  - ./process：后端Pass，如寄存器分配等
  - ./utils：后端需用到的辅助数据结构
  - ./MipsBuilder.java：MIPS构建的控制、辅助类
  - ./Mips*.java：MIPS构建有关的数据结构

3 词法分析

在这一部分中，我们需要将输入的源程序转换为Token串，便于接下来语法分析的进行。此外，在出现错误的情况，我们需要进行记录，并对某些错误情况进行特定的错误处理。

本实现的字符读取、识别过程，**通过超前查看实现，不使用反压字符回文件的操作**。

3.1 总体设计

首先，我们定义以下类：

- **FileLoc**：数据类，记录Token所在的文件位置（行，行内偏移）；
- **Token**：数据类，记录Token类型，Token内容和Token所在的行内字符位置，并提供属性有关的查询、比较等方法；分类通过定义枚举类**TokenType**完成。
- **TokenStream**：存储当前已成功读入的Token，并根据后续的语法分析需求，提供取出Token的**poll()**等方法；
- **Lexer**：将输入程序处理为Token串的逻辑。同时记录分析过程中发生的错误。

编译程序通过实例化**Lexer**类，调用**analyze()**方法进行词法分析。**analyze()**方法的返回值是一个Bool类型，指示词法分析过程中是否出错。若返回值为真，则打印Token串，否则打印错误信息。

需要注意的是，词法分析完成后，无论出现错误与否，均需得到Token串并**继续执行**。是否应停机，交由语法分析部分进行处理。

3.2 方法实现

此处主要对**Lexer**内计划的实现进行介绍。

从文件读取字符的操作，通过创建一个内部类**Reader**实现。**Reader**是对**BufferedReader**的包装。整个读取过程通过**peek()**和**read()**两个操作完成。前者只看当前待读字符，而不从文件中读出，而后者需要从文件中读出。

analyze()方法通过持续调用**Reader::peek()**，判断当前字符应采取的分析子过程并调用，直至文件末尾。

各子过程应调用**Reader::read()**从文件中实际读取字符。子过程通过调用**Lexer::addToken()**和**Lexer::addError()**方法，记录当前过程读取成功的Token,或出现的错误。

当前peek到字符，和执行的子方法对应关系如下：

- 空字符：跳过
- 数字：读取连续的数字串
- 左斜杠：扫描除法符号，或注释
- 双引号：扫描字符串；滤掉两侧的引号，保留引号内的内容。同时应处理转义符引起的识别问题。
- 字母或下划线：扫描标识符，或关键字；连续匹配字符或下划线。
- 其他：扫描符号。

对于scanIdentifier()，在匹配完成后，应先检查给定的keywords集合，判断是否读入了关键字，否则才应将当前读入归为标识符Token。

对于scanSymbol()，主要处理Token对于符号不止一个字符的情况；若没有这种情况的匹配，则检查给定的symbols集合，包含当前读入符号则识别成功，否则报错。

3.3 编码后的修改

编码后,对字符串的识别进行了修改。

在初始设计中，我只是将两端的引号去掉，完全保留中间内容，未有将中间转义符翻译为实际的ASCII字符。后发现，如此处理，应导致后续的中间代码/目标代码生成亦需处理字符串转义问题，违反了单一职责原则，遂决定将转义符的处理置于词法分析处完成。

具体而言，scanString()在读到'\'时，应先peek下一个字符，并根据之检查转义字符集合；若存在，则字符串中放入转义好的ASCII字符，否则报错（未知转义符）。

4 语法分析

整体采用带回溯的递归下降法完成。

4.1 总体设计

首先，为了进行代码组织，建立一抽象类**ASTNode**。任何文法中出现的非终结符均需继承这一抽象类。

在 /frontend下建立Parser类，主程序通过与之交互，开始语法分析过程，获取分析完成的CompileUnit，以及分析过程中是否有错误产生。

在 /frontend/syntax下，按照文法中语法成分的不同，分类建立文件夹，存储各个语法成分对应的类。

此时的顶层模块CompileUnit是语法成分，故亦应置于 /frontend/syntax下，继承ASTNode；其应记录当前CompileUnit中的函数定义、主函数定义和全局变量定义。因此也需提供对应的方法，在下层语法成分完成函数定义或全局变量的分析后，在CompileUnit中进行记录。

各个语法成分的分析逻辑，应放在各语法成分内，而非集中在Parser类，避免Parser变得过分臃肿。

4.2 方法实现

首先，继承ASTNode的各个节点，均需提供**返回自身类返回值**的静态build()方法，根据文法定义中的规则，逐层向下调用子成分的build()方法，进行语法分析。

此时我们希望返回值为成分本身的类，且各个语法成分的构建可能需要顶层成分传下来的不同参数，故不在ASTNode中规定抽象的build()方法。

整个语法分析过程，由Parser调用CompileUnit::build()开始。build()方法的其中一个参数是ErrorEntry集合，各层节点通过向其加入ErrorEntry对象，完成错误的记录。

分析过程中Token的读取，通过TokenStream提供的方法实现。提供的方法应包括:

- 检查Token类型；
- 取出Token；
- 按类型取出Token，不符合给定类型则报错。

考虑到回溯是对于Token读取过程而言的，我们选择在TokenStream中完成回溯逻辑的实现。因此，TokenStream需要实现检查点功能。在可能出现回溯时，应建立检查点，记录当前Token读取位置，并不从TokenStream中实际移除读取的Token；回溯过程中，分析逻辑应正常调用TokenStream提供的Token取出方法，来获取Token，减少额外逻辑带来的复杂度。

在回溯发生时，TokenStream应回到检查点位置；在回溯可能消除后，TokenStream应移除检查点，回到正常的工作模式。

此外，注意到需要记录分析过程。考虑到获取Token必须通过TokenStream，整个记录逻辑应在TokenStream内实现。每成功取出一个Token记录一次，各语法成分的build()方法在执行结束后，应调用TokenStream中方法，记录当前成分已完成分析。

4.3 编码后的修改

本次作业在编码后，较初始设计并未发生较大的变动。

5 语义分析

这部分的工作，与其说是分析语义，实际上是建立各语法成分间的符号关系，比如定义/引用变量和函数。

5.1 总体设计

类似词法分析和语法分析，在 /frontend下，应建立一与主程序交互的类（下称Tabulator，因语义分析相当于建立符号表得名）；主程序通过与此类交互，完成语义分析，并得知分析过程中是否出错。 Tabulator类亦需记录当前的scope，当前可见的函数，变量，循环深度等分析过程中需要的信息。

整个实现过程亦需要一个SymbolTable类，执行分析过程中，当前可见符号的符号表维护过程。

至于符号，首先定义一Symbol类，提供查询符号标识符，和符号所在scope的方法； Symbol下派生ValSymbol和FuncSymbol，对应具体值的符号，和函数的符号； ValSymbol下派生ConstSymbol和VarSymbol，对应常量/变量符号。

同样，为了避免分析逻辑过度集中，各语法成分的语义分析过程，通过在各语法成分类中，实现对象的visit()方法完成。 visit()方法执行过程中，会与Tabulator类交互，进行符号的创建或查询。

语法分析过程需要进行类型检查，故需设计一套类型系统。故应有一抽象类DataType，其下派生：

- ArrayType：数组类型；
- BaseType：不可再分的基本类型，其含有位宽等属性。
  其下可派生IntType等具体类型；
- VoidType：主要用于函数返回值。

5.2 编码后的修改

主要是加入对初始值的处理。虽然本次作业并不要求，但为了便于后续中间代码生成的编写，且初值在语义分析时可知，故在编写语义分析时实现了这部分功能。

对于数组和常量/变量的初始化，创建了一抽象类InitType，其下派生Array/ValInitType，记录初始值信息。

对于常量表达式（主要出现在数组维度、常量和数组的初始化），实现了有关语法成分的calc()方法，求得常量表达式的数字值，以进行初始值的建立。

6 代码生成

本实现选择LLVM IR作为中间代码，以MIPS作为最终目标代码。

6.1 LLVM IR的生成

首先，类似语法分析和语义分析， /frontend下需建立一IrBuilder类。主程序与之交互完成中间代码的构建，同时IrBuilder管理整个中间代码构建流程，记录构建过程中所需要的信息，如函数名与LLVM函数对象，或外部函数的对应关系，并提供构建过程中需要的辅助方法。

为减少冗余和不必要的转换，LLVM IR构建中的类型系统，复用了语义分析中已经建立好的类型系统。在语义分析类型系统的基础上，添加了BooleanType、PointerType（指针类型），以适应LLVM IR对分支条件和变量引用的处理方法。

在LLVM IR的概念中，一切皆Value。因此，应有一Value类，函数、全局变量、基本块、指令等均需继承Value类。

注意到，LLVM IR的原始继承关系中，指令继承自User类，而User继承自Value类。在我们的实现中，User下仅有指令类，故我们选择忽略User类的存在，直接让指令类继承Value类，简化继承关系。

LLVM IR的Value需有严格的顺序编号，故须实现一个计数器辅助类，处理Value编号的问题。

随后就是抽象语法树到LLVM IR的转换。延续我们在语法、语义分析中分布式处理的思路，各个对应了IR的语法成分类（举例而言，包含CompileUnit，但不含BType）中，均需实现build()方法，与IrBuilder类交互，完成各ASTNode到中间代码的转换。

构建完成后的数据结构大致如下：

- IrModule：顶层模块，记录了IR所含的函数、全局变量
- GlobalVariable：全局变量类；提供有关构建方法及toLLVM()方法，输出其LLVM IR形式。
- Function：IR函数类，用容器记录内部所含的各IR基本块
- BBlock：基本块类，用容器记录块内所含指令；

- Inst：指令类，下派生各具体指令。各具体指令类需实现toLLVM()方法，输出LLVM IR。

6.1.1 编码后的修改

主要是对于Function/BBlock类中，使用容器的更改。

在初始实现中，我仅使用了简单的LinkedList，进行基本块/块内函数的记录。考虑到后续优化，可能出现边遍历，边增加/删除块或指令的情况，我额外实现了DoublyLinkedList类，即一双向链表。

DoublyLinkedList以其内部类Node为核心类，提供其创建、维护方法，及其正向/反向的Iterator，允许在某个链表节点前/后插入内容，或直接删除当前节点，为后续的优化等过程的遍历过程带来极大的便利。

后续MIPS的生成也利用了这一数据结构。

6.2 MIPS汇编的生成

在本实现中，我们先将LLVM IR翻译为**以虚拟寄存器作为操作数**的MIPS指令，再通过寄存器分配过程，将虚拟寄存器转换为物理寄存器，实现向MIPS代码的翻译。这使得寄存器分配从翻译过程中解耦出来，大大方便了翻译逻辑的编写。

对于LLVM IR的Phi指令，我们规定一MIPS层面的伪指令phi。在翻译过程中，LLVM IR的Phi将被翻译为我们规定的伪指令phi；在执行寄存器分配前，我们通过RemovePhi过程，完成伪指令phi的消除。

寄存器分配和消Phi逻辑的类，均置于backend/mips/process内，类似于LLVM IR的Pass。后续后端优化的Peephole等过程，也计划放在这一位置。

在backend/mips内，亦有一个MipsBuilder类，管理整个构建流程，并提供构建过程的辅助方法。

翻译过程仍然选择不集中在一个类中编写。在backend/instBuilder中，我为每种LLVM IR指令构建了对应的MIPS翻译逻辑。构建过程由顶层模块MipsModule开始，通过调用其中方法，逐层向下构建，最后完成LLVM IR到MIPS的翻译。

整个结构层次为：

- MipsModule：顶层模块，由其开始构建；

- MipsGlobalVariable：对应应存放在.data区的全局变量；
- MipsFunction：对应一个函数；
- MipsBlock：对应一个基本块；
- Instruction：指令抽象类，下派生各具体指令。属性为用到的操作数，需要实现获取操作数，替换操作数，toMIPS()等方法。

对于操作数，本设计亦进行了层次划分。顶层为抽象类Operand，派生VReg/PReg/Immediate，分别代表虚拟/物理寄存器/立即数（标签）。PReg下派生了AReg，即Arch Registers，MIPS架构定义的32个寄存器。至于为何如此，本设计希望在后续优化中，利用CP1中的32个浮点寄存器，处理全局变量、static变量和寄存器分配时溢出的问题。具体还请移步优化文章。

寄存器分配部分，本设计计划先对跨块活跃的全局变量使用图着色分配法，再对块内活跃的局部变量,采用倒序线性扫描+寄存器池的分配方法。

6.2.1 编码后的修改

架构较初始设计并无太大变化，但对寄存器分配的方案做了调整。

具体而言，本设计参考《Building an Optimizing Compiler》的第13章，对寄存器分配的整个策略做了较大调整：

- 全局变量和局部变量，可用的寄存器范围不再分开；
- 溢出全局变量的读写，通过在每个块内换名，交由局部分配来进行；
- 对块内活跃的全局变量做了分类，并在局部分配时复用了可用的全局寄存器；
- 局部变量的分配，先构建块内的冲突图，将度小于全局变量分配后可用寄存器数者入栈；对于栈内变量，采取图着色方法分配颜色，对于栈外的，采用反向线性扫描分配；
- 在反向线性扫描过程无寄存器可用时，通过溢出某一当前活跃变量的方法，强行获取可用的寄存器。

7 代码优化

代码优化主要分为两部分：对于LLVM IR做变形的前端优化，和对生成的MIPS指令做变形的后端优化。

此处我们仅讨论优化有关的架构和代码组织，并不详细阐述优化的具体实现。优化实现有关内容，还请移步优化文章。

7.1 总体设计

对于优化过程，其应该被视为对编译单元，即MipsModule或IrModule的一个变形过程，或Pass。

每个Pass均需提供一run()方法，执行该Pass内的变形，或分析逻辑。主程序通过给出一个实例化了各个Pass的List，并顺序遍历其中各个Pass，执行其run()方法，完成各Pass的执行。

优化的重点在LLVM IR上。在进行优化前，我们需要对其进行活跃变量分析，控制流分析等等，这些Pass选择放在frontend/llvm/analysis内。生成的分析信息应该作为属性，放在对应的语言层次的类中。举例而言，控制流信息应存在Function类中。为了实现这些分析，可能需要创捷额外的数据结构，比如DAG等。

后端可实现的优化就十分有限了。除了对寄存器分配策略进行调整外，额外可称为一个Pass的优化，应只有Peephole一个。

因此，Peephole与RemovePhi和寄存器分配，均放置在backend/mips/process内。

7.2 编码后的修改

编码后并未对这个优化框架做任何修改。

只不过，在引入Mem2Reg优化后，我发现了我的寄存器分配逻辑似乎出现了问题。为了Debug，我实现了将物理寄存器分配前的MIPS指令亦打印出来的逻辑，便于定位寄存器分配发生错误的类型。

最后发现是，Mem2Reg导致了只定义而不使用的全局变量的出现，而我的冲突图构建未能处理这一情况。